Synapse: Build Plan

Artisan Systems. The company brain, built on LangGraph, Graphiti and Neo4j.

1. What we are building

Synapse watches how a company actually works (mail, meetings, docs, databases), turns that activity into a temporal knowledge graph of people, projects, decisions and outcomes, and then does two things with it: answers questions with cited evidence, and warns teams before they repeat a known mistake. Sold as a service, it has to be multi-tenant, permission-aware and cheap to run per customer. Those three constraints shape almost every choice below.

The product is three verbs. **Remember** (continuous ingestion), **Recall** (cited Q&A over the graph), **Foresee** (proactive warnings and expert discovery).

2. Architecture at a glance

Layer	Choice	Why
Ingestion	Native Google connectors in Python (n8n only for prototyping)	Incremental sync, no licensing trouble when sold as a service
Orchestration	LangGraph	Stateful graphs, checkpointing, human-in-the-loop, streaming
Org memory (graph)	Graphiti on Neo4j Community	Temporal facts, provenance, hybrid search out of the box
Source memory	Postgres + pgvector	Raw chunks, embeddings, tenants, ACLs, audit trail
Agent working memory	LangGraph Postgres checkpointer	Per-conversation state, resumable runs
Models	Ollama locally, Gemini free tier while developing	Data residency for customers who need it, zero cost for dev
API	FastAPI + Redis queue (Arq or Celery)	Async ingestion workers, SSE streaming to the UI

Layer	Choice	Why
Frontend	Next.js, TypeScript, Tailwind, shadcn/ui, Cytoscape.js	Fast to build, graph rendering that scales
Observability	Langfuse, self-hosted	Open source, traces stay inside your infra

Three LangGraph graphs run on top of this and share one tool library (graph search, Cypher templates, document fetch, permission filter).

- **Ingestion graph.** Fetch the changed item, normalise it, chunk it, extract entities and relations, resolve them against existing nodes, write to the graph, verify. Low-confidence merges (is this "Rahul S." the same as "Rahul Sharma"?) pause on an interrupt and land in a review queue instead of silently polluting the graph.
- **Query graph.** Route the question, run hybrid retrieval (semantic, keyword, graph traversal), filter by the asker's permissions, synthesise an answer with citations back to the source email, doc or meeting. If the evidence is thin it says so.
- **Foresight graph.** Triggered when a new project plan, decision or proposal appears, plus a scheduled sweep. It compares the new item against past failures, runs multi-hop traversals (person, project, failure, cause), scores the match, and only raises an alert when the evidence chain clears a threshold. Every alert shows the chain that produced it and has a useful / not useful button that feeds back into the thresholds.

3. Memory layer: what to use

Use Graphiti as the memory engine. It is open source (Apache 2.0), already speaks Neo4j, and solves the three problems you would otherwise rebuild by hand. First, facts have validity windows, so when "Priya owns Atlas" changes, the old edge is invalidated rather than duplicated or deleted, and you can ask what was true on any date. Second, every fact traces back to the episode (email, doc, meeting) that created it. Third, search combines embeddings, BM25 and graph traversal without calling an LLM at query time, which keeps answers fast and cheap. You define the company ontology as Pydantic models (Person, Project, Decision, Problem and so on) and Graphiti extracts to them.

How the alternatives compare:

Option	Verdict
Graphiti	Best fit for cross-source organisational facts that change over time. Cost: ingestion is LLM-heavy and needs a model with reliable structured output.
Mem0	Great for per-user assistant memory (preferences, chat history). Not shaped for company-wide facts across sources. Keep it in mind for a personal-assistant add-on later.
Cognee	Capable pipeline, but I would not bet the temporal side on it. Worth watching.
Hand-rolled (Ollama + Cypher, current approach)	Full control, but you rebuild entity resolution, deduplication and invalidation yourself. Fine for the hackathon demo, wrong for a product.

Keep your own Cypher for Foresight on top of the Graphiti-written graph. That part is your differentiator, so own it.

Three memory tiers. Working memory is the LangGraph state and checkpointer for one conversation. Organisational memory is the Graphiti graph in Neo4j. Source memory is raw chunks and pointers to original documents in Postgres. Answers always cite down to the source tier.

4. The free stack, and which API for what

Database answer: Neo4j Community Edition for the graph, Postgres with pgvector for everything else. Drop ChromaDB. Neo4j has native vector indexes and Postgres covers chunks, auth tables and checkpoints, so Chroma is one more moving part for no gain.

Need	Free option	Notes
Graph DB	Neo4j Community in Docker. Aura Free for demos	Community has one database and no clustering or fine-grained roles, so isolate tenants with Graphiti's group id plus an ACL check in your own code. Dedicated container per enterprise tenant later

Need	Free option	Notes
Relational + vector	Postgres + pgvector in Docker. Supabase free tier for early dev	Also holds the LangGraph checkpointer
Extraction LLM (dev)	Gemini API free tier (Flash or Flash-Lite). Groq free tier for Llama-class models	Rate limited. Free tiers may use your prompts for improvement, so use synthetic or your own data only, never customer data
Extraction LLM (local)	Ollama with a Qwen or Llama model, 14B class if your hardware allows	Models under about 7B miss relations and break structured output. Test before committing
Embeddings	nomic-embed-text or bge-m3 via Ollama	Free and local. Pick one per tenant, because switching means re-embedding everything
Reranking (optional)	bge-reranker, local	Cheap accuracy boost on Q&A
Tracing and evals	Langfuse self-hosted	Keeps prompts and traces in your infra
Queue and cache	Redis	Ingestion jobs, rate limiting
Hosting for demos	Oracle Cloud Always Free ARM VM for backend and databases. Vercel or Cloudflare Pages for the frontend	Generous enough to run the whole stack minus GPU models

Write the model layer behind one interface (LangChain's chat model init or LiteLLM) so a tenant can run on local Ollama, a hosted API, or NVIDIA NIM or vLLM for production inference without touching agent code. Cache extraction by content hash so unchanged documents never cost a second call.

5. Graph schema and provenance

Nodes: Person, Team, Project, Meeting, Document, EmailThread, Decision, Problem, Outcome, Topic, Customer. **Edges:** ATTENDED, OWNS, AUTHORED, DECIDED, CAUSED, FAILED_DUE_TO, RESOLVED_BY, MENTIONS, SUPERSEDES, EXPERT_IN.

Every edge carries source id, the supporting text span, timestamp, extraction confidence, and valid-from and valid-to dates. Every fact also inherits the access control list of the source it came from, so a fact extracted from a private email is only retrievable by people who could read that email. This is the single most important enterprise feature. Get it right in the first version, not later.

6. Build phases

Each phase ends with a working demo and a check that must pass before the next begins.

1. **Foundation.** Docker Compose with Neo4j, Postgres, Redis, FastAPI, Langfuse. Tenant and user tables. Model interface with Ollama and Gemini behind it. *Exit:* one command brings up the full stack.
2. **Ingestion and memory core.** Gmail, Calendar and Drive connectors with incremental sync (Gmail history ids, Calendar sync tokens, Drive changes feed). The ingestion LangGraph, Graphiti writes, ontology models, entity resolution with a review queue. *Exit:* a seeded test company produces a graph where hand-checked facts are correct.
3. **Recall agent.** Query graph, hybrid retrieval, permission filter, cited answers, streaming endpoint. *Exit:* a golden set of questions with known answers scores above your bar on correctness and citation accuracy.
4. **Frontend.** Ask page, graph explorer, entity pages, source manager. Detail in section 7. *Exit:* a non-technical person can connect a workspace and get a useful answer unaided.
5. **Foresight.** Failure-similarity matching, multi-hop traversals, expert discovery, alert feed with evidence chains and feedback buttons. *Exit:* replayed historical projects trigger the warnings they should, and the false-positive rate is low enough that you would not mute it.
6. **Service hardening.** Multi-tenancy, SSO, audit log, rate limits, usage metering, backups, deletion on request, deployment packaging. Detail in section 8.

Build the evaluation harness in phase 2 and keep running it forever. Track extraction precision and recall against labelled documents, entity-merge accuracy, answer correctness and groundedness. Without this you cannot tell whether a model or prompt change helped.

7. Frontend

React with Next.js, streaming answers over server-sent events from LangGraph. Use Cytoscape.js for the graph view in the MVP (rich styling, good layouts, MIT licensed) and move to Sigma.js with WebGL once tenants pass a few thousand visible nodes.

- **Ask.** Chat with inline citations that open the source in a side panel. Shows which graph paths supported the answer.
- **Explorer.** Search an entity, expand its neighbourhood, filter by time with a slider, click any edge to see the source evidence.
- **Entity pages.** A person, project or decision with its timeline, related failures, who knows about it, and open warnings.
- **Foresight feed.** Alerts ranked by confidence, each with its evidence chain, dismiss and useful buttons.
- **Sources and review.** Connect and monitor sync, and a queue for merges and extractions the system was unsure about.
- **Admin.** Members, roles, permissions, usage, audit log.

8. Making it a service

- **Tenancy.** Every node and row carries a tenant id, enforced in one data-access layer rather than scattered through queries. Enterprise tier gets its own Neo4j container and database credentials.
- **Google access.** For Workspace customers, use domain-wide delegation granted by their admin with read-only scopes. For a public app, Gmail's restricted scopes trigger Google's verification and an external security assessment, so budget for that before promising a self-serve signup.
- **n8n licensing.** n8n's Sustainable Use License restricts reselling it as part of a hosted service. Fine for prototyping, but read the terms and replace it with your own connectors before selling. Same exercise for any other component you ship, including Neo4j Community's GPL terms if you distribute it inside a customer-installed package.
- **Security.** Encryption at rest and in transit, secrets in a vault, per-tenant keys later, full audit trail, a data deletion path that removes graph nodes, chunks and embeddings together, and no customer data ever sent to a free-tier API.
- **Deployment modes.** Managed multi-tenant SaaS for speed of adoption. Private single-tenant deployment in the customer's cloud with a local model for regulated buyers. The second is the real enterprise sale, so package it (Docker Compose first, Helm chart

later) from the start.

- **Auth.** Supabase Auth or Keycloak to start. Keycloak when SAML SSO and SCIM provisioning come up.
- **Pricing.** Per connected seat plus a volume component on ingested items, with a flat fee for private deployment. Meter usage from day one so pricing can change without re-engineering.

9. Add-ons for later

- More sources: Slack, Teams, Jira, GitHub, Notion, Confluence, CRM, and meeting transcription from Meet or Zoom through Whisper.
- **MCP server** exposing the company memory so other agents and tools can query it. This turns Synapse from an app into infrastructure.
- Slack and Teams bot for asking and receiving alerts where people already are.
- New-hire briefings and exit interviews that capture a leaving employee's knowledge into the graph.
- Decision log with automatic follow-up when a decision's assumptions stop holding.
- Weekly digest per team, and per-user personalisation through Mem0.
- Custom ontology per company, fact decay and reconciliation jobs, contradiction detection across sources.
- Fine-tuned small extraction models trained on corrected data from the review queue, to cut cost and run well on modest hardware.

10. Risks that will actually hurt

Entity resolution is the hardest problem in the product. Bad merges poison the graph quietly. Keep merge thresholds conservative, route doubtful cases to review, and log every merge so it can be undone.

False alarms kill trust in Foresight faster than anything. Show evidence for every alert, start with high thresholds, and learn from the feedback buttons.

Permission leaks end the company. Test the access filter harder than any other component and make it impossible to bypass from agent code.

Extraction cost and quality at scale. Cache by content hash, use a cheap model for a first pass and a stronger one only on ambiguous items, and measure with the evaluation harness before switching models.